representations of signed integers

sign-magnitude

- one bit used for sign (0 – positive, 1 – negative)
- remaining bits give absolute value of number in binary
- e.g. -3 would be represented as 10 ... 011
- with n bits, range is $-(2^{n-1} - 1)$ to $2^{n-1} - 1$
- disadvantages: two representations of zero; hardware circuits implementing arithmetic operations are relatively inefficient

one's complement

- representation of non-negative integers same as with unsigned
- get representation of -x from that of x by changing all 1s to 0s and 0s to 1s
- e.g. -3 would be represented as 11 ... 100
- still two representations of zero, but more efficient arithmetic implementations!

(to add two one's complement numbers, add them using ordinary binary addition, and then if there is a carry out from the leftmost bit position, add 1 … try it!)

- not used currently for computer system representation of signed integers, but does have an application in Internet communication, for "packet checksums" …

two's complement: representation of signed integers that is currently used in computer systems

- representation of non-negative integers same as with unsigned
- get representation of -x from that of x by changing all 1s to 0s and 0s to 1s, and then adding 1
- e.g. -6 would be represented as

  00 ... 0110 ⇨ 11 ... 1001 + 1 = 11 ... 1010

  above method for finding -x also works for negative x!

  11 ... 1010 ⇨ 00 ... 0101 + 1 = 00 ... 0110
- negative numbers all have 1 as left-most bit (same as with sign-magnitude and one's complement)
- with n bits, range is -2^{n-1} to $2^{n-1} - 1$
- now only one representation of zero, and efficient arithmetic implementations

example of two's complement representation when $n = 4$ (when have just 4 bits to represent a number)

0000 represents 0	1000 represents -8
0001 represents 1	1001 represents -7
0010 represents 2	1010 represents -6
0011 represents 3	1011 represents -5
0100 represents 4	1100 represents -4
0101 represents 5	1101 represents -3
0110 represents 6	1110 represents -2
0111 represents 7	1111 represents -1

addition with two's complement numbers: can just use ordinary binary addition

```
  00...0100    (4)
+ 11...1101    (-3)
  00...0001    (1)
```

note that any carry out of the left-most bit position gets discarded

subtraction of two's complement numbers: find the negative of the number you're subtracting, and add

$$x - y \Rightarrow x + (-y)$$

(remember how to find negative – flip all bits, add one …)

… but, need to be aware of possibility of overflow

overflow is when correct result is not representable in the available number of bits

two kinds of overflow: unsigned overflow, and signed overflow

e.g. suppose have just 4 bits to represent a number

```
  1000
+ 0011
  1011   neither signed nor unsigned overflow
```

```
  0101
+ 0101
  1010   signed overflow
```

```
  1101
+ 0011
  0000   unsigned overflow
```

```
  1100
+ 1011
  0111   both signed and unsigned overflow
```

what is done when overflow occurs?

in RISC-V, nothing automatically … up to program to check for overflow if want such a check

example: checking for *unsigned* overflow of addition operation

```
add s0, s1, s2   # want to check this for unsigned overflow
bltu s0, s1, overflow       # why does this work?
```

example: checking for *signed* overflow of addition operation:

```
add s0, s1, s2   # want to check this for signed overflow
slti t0, s2, 0      # t0 ← 1 if s2 < 0, otherwise t0 ← 0
slt t1, s0, s1      # t1 ← 1 if s0 < s1, otherwise t1 ← 0
bne t0, t1, overflow       # why does this work?
```

how could you check for (signed, unsigned) overflow on *subtractions*?

RISC-V machine language

(reference: text Sections 2.5, 2.6, 2.10)

simplified model of instruction execution

repeat forever:

- load instruction whose address is in PC into IR
- decode
- execute instruction
- increment PC by instruction length

the PC ("program counter") holds memory address of instruction to be executed
the IR ("instruction register") holds the instruction being executed

above simple model suggests that implementing a fast processor will be easier if we have:

- fixed length instructions
- compact instructions
- only a few instruction formats with consistent field locations